2022 3rd International Conference on Computer Vision, Image and Deep Learning & International Conference on Computer Engineering and Applications (CVIDL & ICCEA)

Smart Contract Vulnerability Detection Based on Deep and Cross Network

Nianfeng Li
College of Computer Science and Technology
Changchun University
Changchun, China
linf@ccu.edu.cn

Yang Liu
College of Cyber Security
Changchun University
Changchun, China
210701248@mails.ccu.edu.cn

Lina Li *
College of Computer Science and Technology
Changchun University
Changchun, China
* Corresponding author: liln@ccu.edu.cn

Yuying Wang
College of Cyber Security
Changchun University
Changchun, China
djhwyy@163.com

***Abstract*—Smart contract is the key core part of the blockchain's operation, which has the feature that it cannot be changed after deploying, resulting in the fact that the vulnerabilities associated with smart contracts cannot be fixed. Currently, deep learning has the best performance for detecting smart contract vulnerabilities. However, the common deep learning method treats the smart contract source code as a text sequence, ignoring the running logic and data flow of the program, and can not extract the constructed features. In this paper, a modular vulnerability detection model Link-DC is proposed. the model takes the contract graph constructed by smart contracts and the pattern features defined by expert knowledge as the original data input to capture richer feature information through stitching. Then, the deep and cross network is adopted to construct low-dimensional and sparse features into high-dimensional nonlinear features, thus improving the performance and efficiency of detection. Extensive experiments are conducted on smart contracts with vulnerabilities on the Ether and VNTChain platforms. The experiment results show that the high detection accuracy performance of the Link-DC model is obtained, where the accuracy of detection of re-entry vulnerability, timestamp-dependent vulnerability, and infinite loop vulnerability reaches 94.37%, 92.11%, and 85.29%, respectively.**

Keywords-smart contracts; vulnerability detection; deep learning; feature interaction; cross network

I. INTRODUCTION

Smart contracts make the blockchain highly programmable, but the code of smart contracts cannot avoid some defects in the development process[1]. Meanwhile, smart contracts cannot be changed once they are deployed, resulting in vulnerabilities that cannot be fixed, which poses a security risk to the blockchain network[2]. Common vulnerabilities in smart contracts include integer overflow, timestamp dependency attack, invocation depth attack, miss-operation exception attack, and reentrant attack[3]. These vulnerabilities have become an important factor restricting the development of blockchain. Therefore, it is essential to effectively check the vulnerabilities of smart contracts before deployment.

The current methods for smart contract vulnerability detection are mainly divided into four categories: formal verification methods, symbolic execution methods, fuzzy testing methods, and deep learning methods. The traditional vulnerability detection methods have problems such as long detection time and accuracy rate. The deep learning method is currently a way to improve efficiency and automate detection. SaferSC[4] constructed an Ethernet opcode sequence model using LSTM network. ReChecker[5] implemented vulnerability detection using bidirectional long short-term memory model (BLSTM) and attention mechanism[6]. CGE[7] constructed a directed contract graph through smart contracts and also combined expert knowledge for vulnerability detection innovation.

The deep learning approach greatly improves the efficiency of vulnerability detection, but there is still room for improvement in terms of precision. In this paper, we propose a modular vulnerability detection model Link-DC. In this model, the deep learning network is the basic network structure for detecting smart contract vulnerabilities, and contract graph features[8] and expert security pattern features are taken as the input of the network. Further, we use the deep crossover network to learn nonlinear higher-order features from the combination of contract graph features and expert pattern features to efficiently capture the interaction features with multiple dimensions, without manual feature engineering or traversal search, and with low computational cost.

II. PROBLEM STATEMENT

In this paper, we detect three types of vulnerabilities in smart contracts, namely, reentrant vulnerability, timestamp-dependent vulnerability, and infinite loop vulnerability. The external call of smart contract may be hijacked by the attacker, forcing the contract to execute additional malicious code through the fallback function to achieve compromise, which is called Re-entry vulnerability. When the smart contract uses the block timestamp as the trigger condition to perform some key operations, there will be a timestamp dependency vulnerability. Infinite loop vulnerability means that a program may contain a loop that has no exit condition or cannot meet the exit condition.

The contract graph and the expert pattern feature are used as the input for vulnerability detection. The contract graph is constructed from the control flow and data flow of the smart contract source code, and the key variables and function calls

978-1-6654-5911-2/22/$31.00 ©2022 IEEE

2022 3rd International Conference on Computer Vision, Image and Deep Learning & International Conference on Computer Engineering and Applications (CVIDL & ICCEA) | 978-1-6654-5911-2/22/$31.00 ©2022 IEEE | DOI: 10.1109/CVIDLICCEA56201.2022.9824581

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:43:15 UTC from IEEE Xplore. Restrictions apply.

are graph nodes, by capturing their trajectories executed in time as directed edges[9]. The expert pattern features are generated by defining a sub-pattern one-hot encoding, whose process also involves the important conditions that generate vulnerabilities[10]. Combining these two parts, we improve the data processing ability to obtain high-dimensional feature vectors and improve the accuracy of model detection. The input features are represented as graph feature vector C_r and expert pattern vector S_r. Since the input features are low-dimensional and sparse vectors, they are converted into embedding vectors $E = [e_1, \dots \dots, e_M]$ by Embedding. deep & cross network consists of deep part and cross part, the output vector of deep part is d_o; the output vector of cross part is c_o, the fusion vector is defined as D_o, and the output of the whole model is Q_{mlp}.

Our goal is to improve the accuracy of smart contract vulnerability detection by deep & cross network module. The specific process is as follows: the feature crossover is performed on the S_r, and C_r input vectors to obtain the high-dimensional nonlinear feature D_o. Then D_o is outputted to the detection result by MLP. For the whole feature crossover model, the input is $S_r \cup C_r$ and the output is Q_{mlp} and determines whether there is a specific vulnerability.

III. Detection Model Based on Feature Crossing

A. Overall architecture

In this paper, a feature crossing (Link-DC) model based on the deep & cross network is proposed, and the architecture of this model is shown in Fig.1. The structure has two features: (1) the original sparse vectors are mapped into the low-dimensional space by a neural network. (2) the feature cross vectors are obtained by a DCN module. The DCN module constructs the high-order vectors directly using the cross and deep network, then fuses the output vectors of the two networks, finally outputs the detection results by a fully connected layer with an activation function.

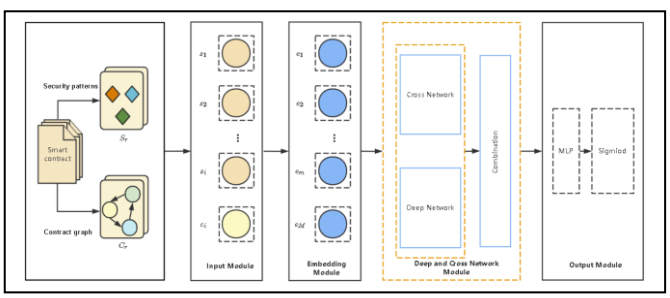

Figure 1. The architecture of smart contract vulnerability detection model.

B. Data embedding

In order to achieve uniform processing of different data types, we embed the input vector S_r with C_r's by mapping as $E = [e_1, \dots \dots, e_M]$. Because one-hot encoding will produce sparse high-dimensional feature data, only the indexes corresponding to non-zero features are stored, which is not conducive to the model for training. We embed these features into a low-dimensional space to represent each category feature with a low-dimensional vector.

Both discrete features and continuous features belong to a separate feature domain. In the Embedding layer, these three types of single-valued discrete features, multi-valued discrete features and continuous features are handled differently. For single-valued discrete features, the corresponding embedding vector is obtained directly from the embedding word list as follows:

$$e_i = V_i x_i \tag{1}$$

where V_i is the embedding matrix corresponding to the feature group, and x_i is the one-hot encoded representation vector of features in the feature group.

For multi-valued discrete features, after the corresponding embedding vectors are obtained from the embedding word list, the Embedding of the same feature domain needs to be averaged by avg-pooling.

$$e_i = \frac{1}{q} V_i x_i \tag{2}$$

where q is the number of values taken in the multi-valued discrete feature. For discrete features, the above x_i is a one-hot vector or multi-hot vector, which takes the value either 0 or 1.

For continuous features, we multiply the scalar value directly with its corresponding embedding vector.

$$e_m = v_m x_m \tag{3}$$

where v_m is an embedding vector and x_m is a scalar value.

C. Deep and Cross Network

The core architecture of deep and cross network is shown in the Fig.2. The data are fed into cross network and deep network in parallel after the embedding process, and the input features are fused by the combination layer to get the output.

The one-hot category feature encoding is used in the expert model, which makes the input features very high dimensional and sparse, and the embedding process can reduce the dimensionality of the input into a valuable feature vector. The continuous features need to be normalized and stacked together with the embedding vectors to get the original input vectors.

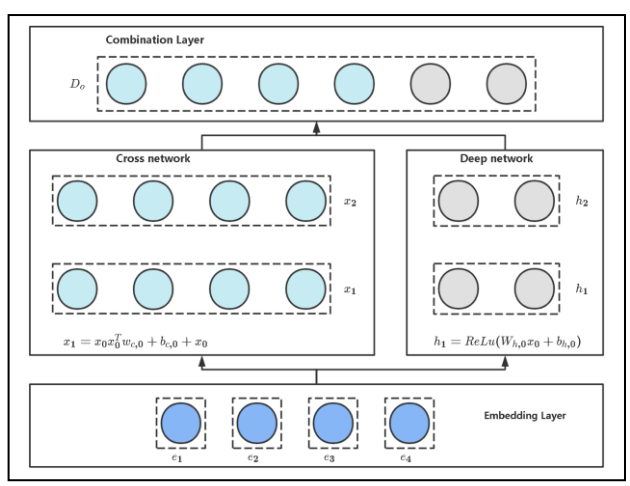

Figure 2. The core structure of deep and cross network.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:43:15 UTC from IEEE Xplore. Restrictions apply.

Crossover networks are the core part of DCNs that efficiently learn combinatorial features to obtain high-dimensional feature vectors in the following form.

$$f(x_l,\ w_l,\ b_l) = x_0 x_l^T w_l + b_l \tag{4}$$

$$x_{l+1} = f(x_l,\ w_l,\ b_l) + x_l \tag{5}$$

where x_l and x_{l+1} are the outputs of the l th and (l + 1)th cross layer, respectively, and w_l and b_l are the connection parameters between these two layers. The $f\ (xl, wl, bl)$ is the residual between the output of the layer and the output of the previous layer when fitted. The special network structure of the cross network makes the order of the cross feature increase as the layer depth increases. The order of the cross feature of layer l cross network is l +1 with respect to the input x_0.

Complexity analysis. Assume that there is a total of L_c cross layer and the starting input x_0 has dimension *d*. Then the number of parameters of the whole cross network is as follows.

$$O = d * L_c * 2 \tag{6}$$

Each layer of *W* and *b* is of *d* dimensions. From the above equation, we can find that the complexity is a linear function of the input dimension *d*. So the complexity introduced by the cross network is insignificant compared to the deep network. This ensures that the complexity of the DCN is of the same level as the DNN[11]. The paper[12] states that the reason why cross network can learn combinatorial features efficiently is that the rank of $x_0 * x_T$ is 1, which allows us to get all the cross terms without computing and storing the whole matrix.

Due to the relatively few parameters of the cross network, its expression ability is limited. In order to learn the highly nonlinear combination features, DCN introduces the deep network forward propagation of fully connected neural network in parallel[13].

IV. Experiment evaluation

In this section, we will evaluate the performance of the proposed link DC model. The purpose of our experiment is to answer the following questions, that is, how about the performance of link DC model compared with CGE (the highest accuracy of smart contract vulnerability detection at present).

A. Datasets

We conducted experiments on two datasets of smart contracts, ESC (Ethernet smart contracts) and VSC (VNT smart contracts), which are collected from real projects on the Ether and VNT chain platforms, respectively. For the ESC dataset[14], we conduct experiments to detect re-entry and timestamp dependency vulnerabilities, while infinite loop vulnerabilities are tested on the VSC dataset[15].

ESC dataset. This dataset contains about 307,815 function functions from 41,072 ethereum smart contracts. In these functions, the use of *call.value* is involved, thus increasing the risk of reentrant vulnerability attacks. Functions containing *block.timestamp* statements are vulnerable to timestamp dependency vulnerabilities. The presence of a loop statement in a function causes the program to fail to stop execution.

VSC dataset. In this dataset, there are 13,915 functions collected from 4,672 smart contracts in the VNT Chain platform. This is a public blockchain platform running smart contracts written in a Class C language , which is initiated by companies and universities in Singapore, China, and Australia.

B. Parameter settings

In cases not specifically mentioned in this paper, we set the default settings for the neural network model parameters as follows: learning rate l=0.0001, batch size=128, dropout=0.1. For each data set, we randomly divided it into training set and test set according to the ratio of 4:1, and reported the average results. The basic fact labels of contract functions are provided by experts.

C. Performance analysis

The performance of different models on ESC dataset and VSC dataset is shown in the table 1. Testing on re-entry vulnerabilities and timestamp-dependent vulnerabilities on Ethernet smart contracts, Link-DC has a significant improvement in accuracy, recall, precision, and F1 score. In addition, there is a slight improvement in accuracy and precision for testing against infinite loop vulnerabilities on the VSC dataset.

Re-entry vulnerabilities detection. Link-DC achieves 94.37% accuracy and 88.7% precision in reentrant vulnerability detection, which has 5.22% improvement in accuracy and 3.46% improvement in precision compared to CGE, respectively. The recall and F1 scores reach 95.57% and 92.01%, respectively, an improvement of 7.95% and 5.6%

TABLE I. Performance comparison of different model

Method	Reentrancy（ESC dataset）				Timestamp（ESC dataset）				Loops（VSC dataset）			
	Acc (%)	Recall (%)	Precision (%)	F1 (%)	Acc (%)	Recall (%)	Precision (%)	F1 (%)	Acc (%)	Recall (%)	Precision (%)	F1 (%)
CGE	89.15	87.62	85.24	86.41	89.02	88.10	87.41	87.75	83.21	82.29	81.97	82.13
Link-DC	94.37	95.57	88.7	92.01	92.11	89.85	90.95	90.40	85.29	69.68	83.09	75.28

compared to CGE. All metrics are based on the fact that if one call to *call.value* is made in a smart contract, there is a possibility of being affected by a re-entry attack, and functions that are not known to have a call to *call.value* are not affected by the re-entry vulnerability.

Timestamp-dependent vulnerabilities detection. Link-DC achieves 92.11% accuracy and 90.95% precision, an improvement of 3.09% and 3.54% in accuracy and precision, respectively, compared to CGE. The recall and F1 scores reached 89.85% and 90.40%, an improvement of 1.75% and

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:43:15 UTC from IEEE Xplore. Restrictions apply.

2.65% compared to that of CGE, respectively. The recall rate has some improvement in the detection of timestamp-dependent vulnerabilities, but the effect is not as obvious as the detection of re-entrant vulnerabilities, indicating that the improvement is not the same for different types of vulnerabilities. The lack of fine-grained details of data dependency and control dependency in the process of contract mention generation leads to insignificant performance improvement.

Infinite loop vulnerabilities detection. Link-DC achieves an accuracy of 85.29% and a precision of 83.09%, which is an improvement of 2.08% and 1.12% compared to CGE. On the contrary, the recall reaches 69.68% and the F1 score reaches 75.28%, a decrease of 12.61% and 6.85% respectively compared to CGE. This situation may be due to problems in the labeling of the original data, and the incorrectly labeled samples need to be corrected by means of software and manual auditing. Second, it may also be due to the imbalance between the number of positive and negative samples, and the existence of a circular body structure in many programs, which leads to an infinite loop vulnerability in the error judgments.

V. Conclusion and Future Work

In this paper, we propose a new smart contract vulnerability detection model, called as Link-DC. In this model, the deep and cross network is adopted to construct high-order nonlinear features, and these features are output to a fully connected layer to product the detection result. This model can fully and efficiently extract the features of the original data, thus improving the performance of the deep learning model and the training efficiency of the model. We conduct experiments on two public datasets, and the results show that Link-DC outperforms the best CGE method in terms of correctness and various performance metrics for re-entrant vulnerabilities, timestamp-dependent vulnerabilities, and improves accuracy and precision in the detection of infinite loop vulnerabilities. In addition, we use a modular structure to design Link-DC, thus it is highly flexible and scalable, and can be well integrated with other models. In terms of feature construction, different feature intersection methods are adopted to construct feature vectors. There are still some future works to be improved.

Acknowledgment

This work was supported by the Natural Science Foundation of Jilin Province (No.YDZJ202101ZYTS191), and in part by the Jilin Scientific and Technological Development Program (No.20210201083GX), and the Industry University Research Innovation Fund of Chinese university (No.2020HYB03002).

References

[1] Panda S K, Satapathy S C. An Investigation into Smart Contract Deployment on Ethereum Platform Using Web3. js and Solidity Using Blockchain[M]//Data Engineering and Intelligent Computing. Springer, Singapore, 2021: 549-561.

[2] Wang Z, Jin H, Dai W, et al. Ethereum smart contract security research: survey and future research opportunities[J]. Frontiers of Computer Science, 2021, 15(2): 1-18.

[3] NI Y, ZHANG C, YIN T. A Survey of Smart Contract Vulnerability Research[J]. Journal of Cyber Security, 2020, 5(3): 78-99.

[4] Tann W J W, Han X J, Gupta S S, et al. Towards safer smart contracts: A sequence learning approach to detecting security threats[J]. arXiv preprint arXiv:1811.06632, 2018.

[5] Qian P, Liu Z, He Q, et al. Towards automated reentrancy detection for smart contracts based on sequential models[J]. IEEE Access, 2020, 8: 19685-19695.

[6] Zhou P, Shi W, Tian J, et al. Attention-based bidirectional long short-term memory networks for relation classification[C]//Proceedings of the 54th annual meeting of the association for computational linguistics (volume 2: Short papers). 2016: 207-212.

[7] Liu Z, Qian P, Wang X, et al. Combining graph neural networks with expert knowledge for smart contract vulnerability detection[J]. IEEE Transactions on Knowledge and Data Engineering, 2021.

[8] Zhuang Y, Liu Z, Qian P, et al. Smart Contract Vulnerability Detection using Graph Neural Network[C]//IJCAI. 2020: 3283-3290.

[9] Allamanis M, Brockschmidt M, Khademi M. Learning to represent programs with graphs[J]. arXiv preprint arXiv:1711.00740, 2017

[10] Jiang B, Liu Y, Chan W K. Contractfuzzer: Fuzzing smart contracts for vulnerability detection[C]//2018 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, 2018: 259-269.

[11] Zhang W, Qin J, Guo W, et al. Deep learning for click-through rate estimation[J]. arXiv preprint arXiv:2104.10584, 2021.

[12] Wang R, Fu B, Fu G, et al. Deep & cross network for ad click predictions[C]//Proceedings of the ADKDD'17. 2017: 1-7.

[13] Hirasawa K, Ohbayashi M, Koga M, et al. Forward propagation universal learning network[C]//Proceedings of International Conference on Neural Networks. IEEE, 1996, 1: 353-358.

[14] "Ethereum," Website, 2015, https://github.com/ethereum/ go-ethereum.

[15] "Vntchain," Website, 2018, https://github.com/vntchain/go-vnt.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:43:15 UTC from IEEE Xplore. Restrictions apply.